

DOCKER TASK - 01

Setup a virtual machine using Ubuntu image.

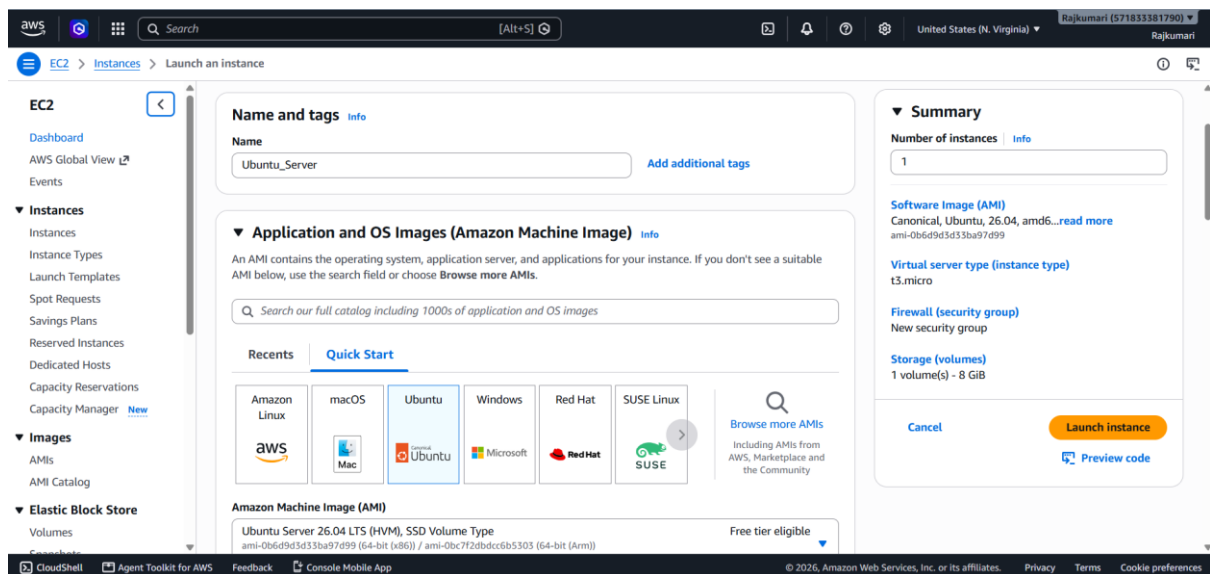
Step 1: Open AWS EC2

1. Login to the **AWS Management Console**.
2. In the search bar, type **EC2**.
3. Open **EC2**.
4. Select your desired AWS Region, for example:

US East (N. Virginia) – us-east-1

5. From the EC2 dashboard, click **Instances**.
6. Click **Launch instances**.

Purpose: EC2 provides the virtual machine/server that will run Ubuntu.



Step 2: Create an SSH Key Pair

Before launching the VM, create a key pair so that you can securely connect to Ubuntu.

Go to:

EC2 → Network & Security → Key Pairs → Create key pair

Enter:

Name: ubuntu-vm-key

Key pair type: RSA

Private key format: .pem

Click **Create key pair**.

The file will download:

ubuntu-vm-key.pem

Keep this file safely on your computer.

Important: Never share the .pem file with anyone.

Purpose: The private key is used for SSH authentication when connecting to your Ubuntu VM.

Step 3: Select the Ubuntu Image

In the **Launch an instance** page, under:

Application and OS Images (Amazon Machine Image)

Select:

Quick Start → Ubuntu

Choose:

Ubuntu Server 24.04 LTS

Make sure the image is an official Ubuntu/Canonical image.

You should see something similar to:

Ubuntu Server 24.04 LTS

64-bit (x86)

Purpose: An AMI (Amazon Machine Image) is the template used to create your EC2 virtual machine. Here, the template contains the Ubuntu operating system.

Step 4: Choose the Instance Type

Under **Instance type**, choose a small instance suitable for learning.

For example:

Instance type: t3.micro

If your AWS console shows another **Free tier eligible** small instance, you can select that instead.

For your assignment, you don't need a powerful server.

For example:

CPU: Small

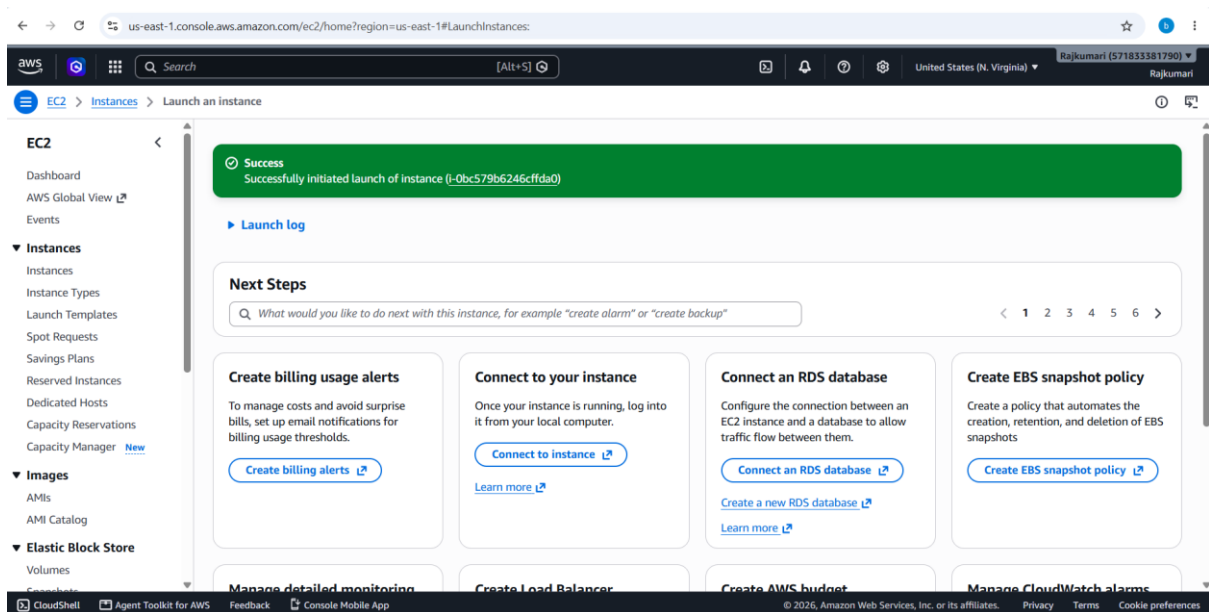
RAM: Small

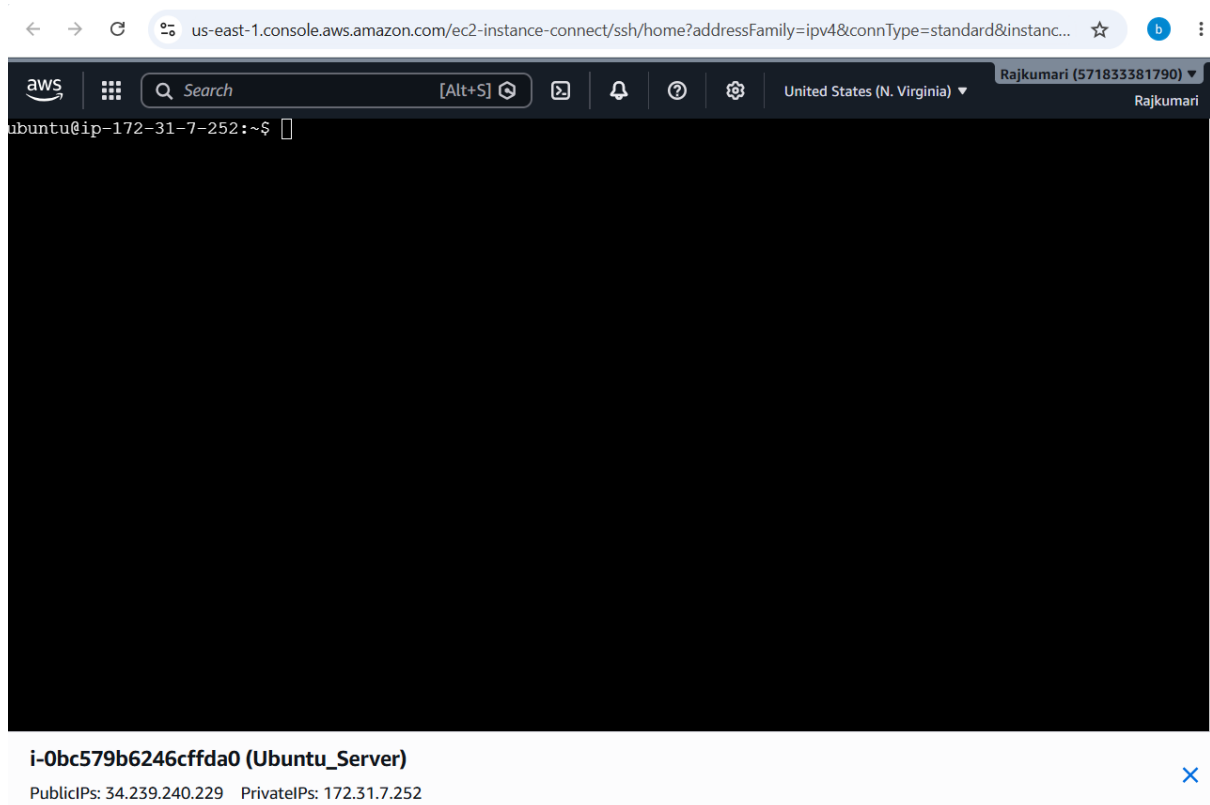
Purpose: Learning / Testing

Then under **Key pair**, select:

ubuntu-vm-key

Purpose: The instance type determines the VM's CPU, memory, and overall compute capacity.





1. INSTALL DOCKER ON EC2

Step 1.1 — Connect to Ubuntu EC2

From your Windows Gitbash:

```
ssh -i "ubuntu-vm-key.pem" ubuntu@YOUR_PUBLIC_IP
```

Once connected:

```
whoami
```

Expected:

```
ubuntu
```

```
ubuntu@ip-172-31-7-252: ~  
brajk@RajKumari MINGW64 ~/Downloads  
$ ssh -i server-2.pem ubuntu@34.239.240.229  
The authenticity of host '34.239.240.229 (34.239.240.229)' can't be established.  
ED25519 key fingerprint is SHA256:hgoFp2KtIGF0cqiHmdCvR9S1EI0hqvGcTn29e9FmgYy.  
This key is not known by any other names  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '34.239.240.229' (ED25519) to the list of known hosts  
.  
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)  
  
* Documentation:  https://docs.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
System information as of Wed Aug 19 08:07:14 UTC 2026  
  
System load:  0.0           Temperature:   -273.1 C  
Usage of /:   11.1% of 18.25GB  Processes:    118  
Memory usage: 26%           Users logged in: 0  
Swap usage:   0%             IPv4 address for ens5: 172.31.7.252  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
  
Last login: Wed Aug 19 08:00:56 2026 from 18.206.107.28  
ubuntu@ip-172-31-7-252:~$
```

Step 1.2 — Update Ubuntu

`sudo apt update`

Then:

`sudo apt upgrade -y`

Step 1.3 — Install Docker

Install Docker using Ubuntu's package manager:

`sudo apt install docker.io -y`

Check Docker:

`docker --version`

You should see something similar to:

Docker version 28.x.x

```

ubuntu@ip-172-31-7-252: ~
Selecting previously unselected package pigz.
(Reading database ... 85303 files and directories currently installed.)
Preparing to unpack .../0-pigz_2.8-1build1_amd64v3.deb ...
Unpacking pigz (2.8-1build1) ...
Selecting previously unselected package bridge-utils.
Preparing to unpack .../1-bridge-utils_1.7.1-4ubuntu3_amd64v3.deb ...
Unpacking bridge-utils (1.7.1-4ubuntu3) ...
Selecting previously unselected package runc.
Preparing to unpack .../2-runc_1.4.0-0ubuntu1_amd64v3.deb ...
Unpacking runc (1.4.0-0ubuntu1) ...
Selecting previously unselected package containerd.
Preparing to unpack .../3-containerd_2.2.2-0ubuntu1.1_amd64v3.deb ...
Unpacking containerd (2.2.2-0ubuntu1.1) ...
Selecting previously unselected package dns-root-data.
Preparing to unpack .../4-dns-root-data_2025080400build1_all.deb ...
Unpacking dns-root-data (2025080400build1) ...
Selecting previously unselected package dnsmasq-base.
Preparing to unpack .../5-dnsmasq-base_2.92-1ubuntu0.4_amd64v3.deb ...
Unpacking dnsmasq-base (2.92-1ubuntu0.4) ...
Selecting previously unselected package docker.io.
Preparing to unpack .../6-docker.io_29.1.3-0ubuntu4.1_amd64v3.deb ...
Unpacking docker.io (29.1.3-0ubuntu4.1) ...
Selecting previously unselected package ubuntu-fan.
Preparing to unpack .../7-ubuntu-fan_0.12.17_all.deb ...
Unpacking ubuntu-fan (0.12.17) ...
Setting up dnsmasq-base (2.92-1ubuntu0.4) ...
Setting up runc (1.4.0-0ubuntu1) ...
Setting up dns-root-data (2025080400build1) ...
Setting up bridge-utils (1.7.1-4ubuntu3) ...
Setting up pigz (2.8-1build1) ...
Setting up containerd (2.2.2-0ubuntu1.1) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/containerd.service'
→ '/usr/lib/systemd/system/containerd.service'.
Setting up ubuntu-fan (0.12.17) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/ubuntu-fan.service'
→ '/usr/lib/systemd/system/ubuntu-fan.service'.
Setting up docker.io (29.1.3-0ubuntu4.1) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service' →
'/usr/lib/systemd/system/docker.service'.
Created symlink '/etc/systemd/system/sockets.target.wants/docker.socket' → '/usr
/lib/systemd/system/docker.socket'.
Processing triggers for dbus (1.16.2-2ubuntu4) ...
Processing triggers for man-db (2.13.1-1build1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-7-252:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
ubuntu@ip-172-31-7-252:~$ |

```

Step 1.4 — Start Docker

`sudo systemctl start docker`

Enable Docker so it starts automatically after reboot:

`sudo systemctl enable docker`

Check status:

`sudo systemctl status docker`

You should see:

Active: active (running)

Press:

q

to exit the status screen.

```
ubuntu@ip-172-31-7-252: ~  
ubuntu@ip-172-31-7-252:~$ sudo systemctl start docker  
ubuntu@ip-172-31-7-252:~$ sudo systemctl enable docker  
ubuntu@ip-172-31-7-252:~$ sudo systemctl status docker  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: e  
   Active: active (running) since Wed 2026-08-19 08:24:29 UTC; 2min 39s ago  
  Invocation: 52384b78c926495ca9f6a53796fa9058  
 TriggeredBy: ● docker.socket  
    Docs: https://docs.docker.com  
   Main PID: 2503 (dockerd)  
     Tasks: 9  
    Memory: 41.1M (peak: 41.2M)  
       CPU: 433ms  
    CGroup: /system.slice/docker.service  
            └─2503 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/cont  
Aug 19 08:24:28 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:28.610390>  
Aug 19 08:24:28 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:28.655064>  
Aug 19 08:24:28 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:28.661441>  
Aug 19 08:24:29 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:29.090317>  
Aug 19 08:24:29 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:29.196887>  
Aug 19 08:24:29 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:29.197077>  
Aug 19 08:24:29 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:29.242301>  
Aug 19 08:24:29 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:29.251141>  
Aug 19 08:24:29 ip-172-31-7-252 dockerd[2503]: time="2026-08-19T08:24:29.251528>  
Aug 19 08:24:29 ip-172-31-7-252 systemd[1]: Started docker.service - Docker App  
ubuntu@ip-172-31-7-252:~$ |
```

Step 1.5 — Allow Ubuntu User to Run Docker

By default, Docker commands may require sudo.

Add the ubuntu user to the Docker group:

```
sudo usermod -aG docker ubuntu
```

Then either log out and reconnect:

```
exit
```

Reconnect:

```
ssh -i "ubuntu-vm-key.pem" ubuntu@YOUR_PUBLIC_IP
```

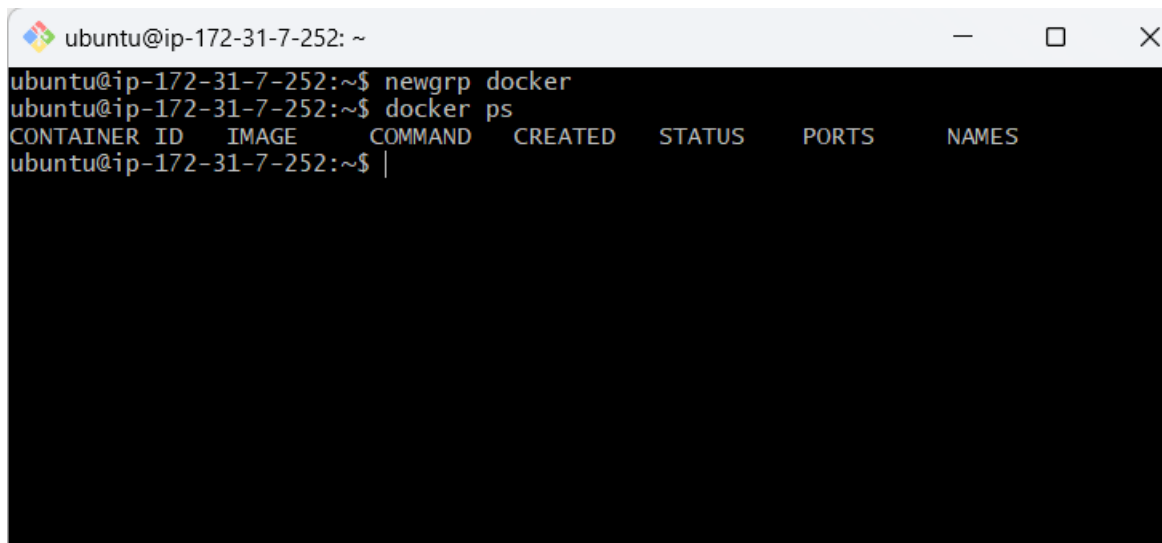
Or run:

```
newgrp docker
```

Now test:

```
docker ps
```

If it works without sudo, Docker is correctly configured.

A terminal window with a title bar showing 'ubuntu@ip-172-31-7-252: ~'. The terminal content shows the user running 'newgrp docker' and 'docker ps'. The 'docker ps' command returns a table with headers: CONTAINER ID, IMAGE, COMMAND, CREATED, STATUS, PORTS, and NAMES. The table is currently empty, showing only the headers.

```
ubuntu@ip-172-31-7-252:~$ newgrp docker
ubuntu@ip-172-31-7-252:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
ubuntu@ip-172-31-7-252:~$ |
```

2. PULL 5 DOCKER IMAGES

Your assignment requires:

1. Nginx
2. Apache Tomcat
3. Ubuntu
4. Jenkins
5. SonarQube

Step 2.1 — Pull Nginx

```
docker pull nginx
```

Docker downloads the official Nginx image from Docker Hub.


```

ubuntu@ip-172-31-7-252: ~
ubuntu@ip-172-31-7-252:~$ newgrp docker
ubuntu@ip-172-31-7-252:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-7-252:~$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
5a4222b844e8: Pull complete
26c307b5e35a: Pull complete
3c55dc422a81: Pull complete
b8b80b9bc028: Pull complete
f5de6e85ac74: Pull complete
d84ae7b21412: Pull complete
c0df8d325117: Pull complete
0f03cb4db0ef: Download complete
92fcf0fc2ef2: Download complete
Digest: sha256:8541484afbc9c8a5a8a99b379568ebbc957f658583ec9448fc43104229c03cf8
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
ubuntu@ip-172-31-7-252:~$ docker image
Usage: docker image COMMAND

Manage images

Commands:
  build      Build an image from a Dockerfile
  history    Show the history of an image
  import     Import the contents from a tarball to create a filesystem image
  inspect    Display detailed information on one or more images
  load       Load an image from a tar archive or STDIN
  ls         List images
  prune      Remove unused images
  pull       Download an image from a registry
  push       Upload an image to a registry
  rm         Remove one or more images
  save       Save one or more images to a tar archive (streamed to STDOUT by default)
  tag        Create a tag TARGET_IMAGE that refers to SOURCE_IMAGE

Run 'docker image COMMAND --help' for more information on a command.
ubuntu@ip-172-31-7-252:~$ docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	8541484afbc9	241MB	66MB	

```

ubuntu@ip-172-31-7-252:~$

```

Step 2.2 — Pull Apache Tomcat

docker pull tomcat

```
ubuntu@ip-172-31-7-252: ~  
ubuntu@ip-172-31-7-252:~$ docker pull tomcat  
Using default tag: latest  
latest: Pulling from library/tomcat  
4f4fb700ef54: Pull complete  
157ff4c8f6ad: Pull complete  
0926a8eb0e60: Pull complete  
c79111691f56: Pull complete  
1cf3a45d29b2: Pull complete  
b2e6c4098a05: Pull complete  
9581ce5d35ae: Pull complete  
1d40032bb7c8: Download complete  
f4456cbc3a39: Download complete  
Digest: sha256:3f89144bffa0cb68ae4eecfe0a817cb319ac4f7f22beb03cd8d6fca0ef75f2dbf  
Status: Downloaded newer image for tomcat:latest  
docker.io/library/tomcat:latest  
ubuntu@ip-172-31-7-252:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	8541484afbc9	241MB	66MB	
tomcat:latest	3f89144bffa0c	589MB	158MB	

```
ubuntu@ip-172-31-7-252:~$ |
```

Step 2.3 — Pull Ubuntu

docker pull ubuntu

```
ubuntu@ip-172-31-7-252:~$ docker pull ubuntu  
Using default tag: latest  
latest: Pulling from library/ubuntu  
f3db1cd94078: Pull complete  
06e9d71331fb: Pull complete  
f79afc4bbefe: Download complete  
Digest: sha256:6df9e8dd1eac389ebfef692c9648449adeb815d01e16e29cd6f3e50fe64ba9a6  
Status: Downloaded newer image for ubuntu:latest  
docker.io/library/ubuntu:latest  
ubuntu@ip-172-31-7-252:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	8541484afbc9	241MB	66MB	
tomcat:latest	3f89144bffa0c	589MB	158MB	
ubuntu:latest	6df9e8dd1eac	160MB	45.3MB	

```
ubuntu@ip-172-31-7-252:~$ |
```

Step 2.4 — Pull Jenkins

For Jenkins, use the official Jenkins image:

docker pull jenkins/jenkins:lts

```

ubuntu@ip-172-31-7-252:~$ docker pull jenkins/jenkins:latest
latest: Pulling from jenkins/jenkins
adea3ef8b4db: Pull complete
b890c9407285: Pull complete
844d30ade61d: Pull complete
8957c4df8f9d: Pull complete
a52ea482db57: Pull complete
f19a62391432: Pull complete
d8ae5f65d97e: Pull complete
ae71a7561ea5: Pull complete
01f97d1e583f: Pull complete
a5e3afd1e0b3: Pull complete
add9a913292e: Pull complete
18e2bde91fb0: Pull complete
2445bf8f759f: Download complete
Digest: sha256:8547df3b0db2803d158ecc9499207a056bb30c23fddc18bb5b4a4dc14e77dd09
Status: Downloaded newer image for jenkins/jenkins:latest
docker.io/jenkins/jenkins:latest
ubuntu@ip-172-31-7-252:~$ docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	In Use
jenkins/jenkins:latest	8547df3b0db2	801MB	290MB		
nginx:latest	8541484afbc9	241MB	66MB		
tomcat:latest	3f89144bfff0c	589MB	158MB		
ubuntu:latest	6df9e8dd1eac	160MB	45.3MB		

Step 2.5 — Pull SonarQube

docker pull sonarqube

```

ubuntu@ip-172-31-7-252:~$ docker pull sonarqube
Using default tag: latest
latest: Pulling from library/sonarqube
4f4fb700ef54: Pull complete
b0370ca8035d: Pull complete
e93fb5720ecd: Pull complete
baedfd75dd2a: Download complete
f1b1a0791ad3: Download complete
Digest: sha256:6287ef13f5265d3eb26e6d5953ee08764d4ca73b3dabcd4e7de79ab040a6dfea
Status: Downloaded newer image for sonarqube:latest
docker.io/library/sonarqube:latest
ubuntu@ip-172-31-7-252:~$ docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	In Use
jenkins/jenkins:latest	8547df3b0db2	801MB	290MB		
nginx:latest	8541484afbc9	241MB	66MB		
sonarqube:latest	6287ef13f526	2.53GB	1.09GB		
tomcat:latest	3f89144bfff0c	589MB	158MB		
ubuntu:latest	6df9e8dd1eac	160MB	45.3MB		

3. Run Nginx Container and Expose Port 81

Now your task says:

Run nginx container and expose on port 81.

There are two ports involved:

EC2 Port 81 → Container Port 80

Nginx normally listens on port **80 inside the container**.

We will expose it through **EC2 port 81**.

Step 3.1 — Run Nginx

```
docker run -d --name nginx-container -p 81:80 nginx
```

Explanation:

docker run

Creates and starts a container.

-d

Runs the container in detached/background mode.

--name nginx-container

Gives the container a name.

-p 81:80

Maps:

EC2 port 81 → Container port 80

nginx

Uses the Nginx image.

```
ubuntu@ip-172-31-7-252: ~
ubuntu@ip-172-31-7-252:~$ docker run -d --name nginx-container -p 81:80 nginx
869b56729b552287ce9e9782c4d27eeb53fecaa7ec28cda9e405bf8437dcb03f
ubuntu@ip-172-31-7-252:~$ CONTAINER ID    IMAGE    PORTS
xxxxxxx    nginx    0.0.0.0:81->80/tcp
CONTAINER: command not found
bash: 80/tcp: No such file or directory
ubuntu@ip-172-31-7-252:~$ docker ps
CONTAINER ID    IMAGE    COMMAND                                     CREATED          STATUS
869b56729b55    nginx    "/docker-entrypoint..."                 About a minute ago Up About
a minute       0.0.0.0:81->80/tcp, [::]:81->80/tcp nginx-container
```

Step 3.3 — Configure AWS Security Group

This step is **very important**.

Your EC2 security group currently probably allows SSH:

TCP 22

You need to allow:

TCP 81

Go to:

AWS Console → EC2 → Instances → Your Ubuntu Instance → Security → Security Groups

Click the security group.

Go to:

Inbound rules → Edit inbound rules → Add rule

Configure:

Type: Custom TCP

Port: 81

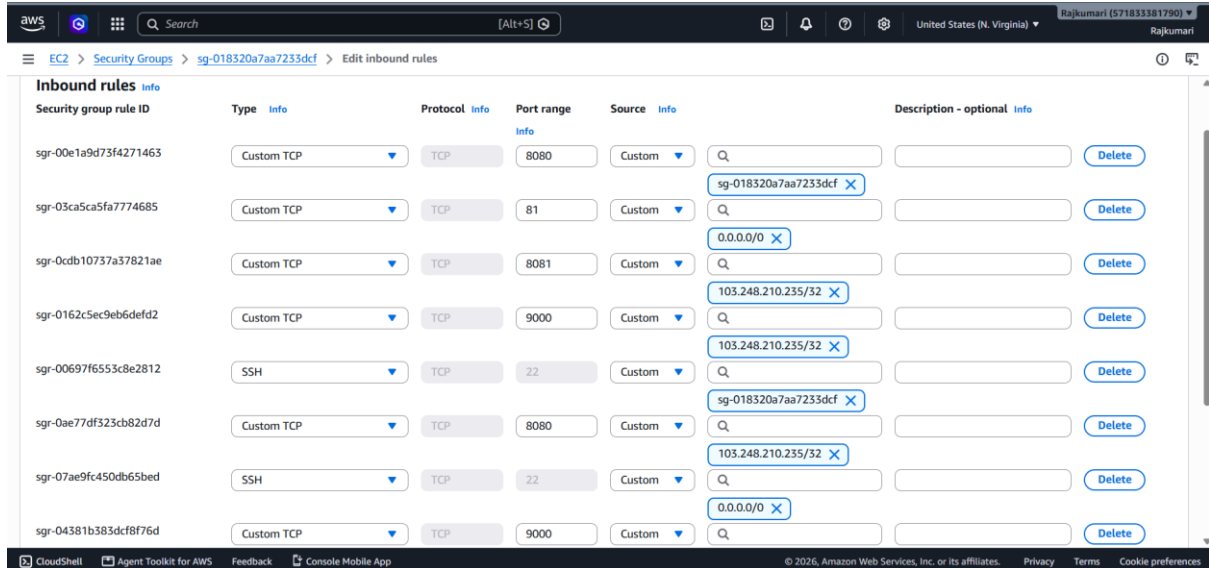
Source: My IP

For temporary testing, you could use:

0.0.0.0/0

but **My IP** is preferable for security.

Save the rule.



Step 3.4 — Test Nginx

Open your browser:

http://YOUR_PUBLIC_IP:81

For example:

http://54.123.45.67:81

You should see:

Welcome to nginx!

Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

4. DELETE APACHE TOMCAT IMAGE

Your assignment says:

Delete the Apache Tomcat image from local.

First check the image:

docker images

Find:

tomcat

Step 4.1 — Remove Tomcat image

Run:

```
docker rmi tomcat
```

If it works, you'll see something like:

Untagged: tomcat:latest

Deleted: sha256:xxxxxxxxx

Step 4.2 — Verify

docker images

Now you should have:

nginx

ubuntu

jenkins/jenkins

sonarqube

```
ubuntu@ip-172-31-7-252:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	In Use
jenkins/jenkins:lts	8547df3b0db2	801MB	290MB		
nginx:latest	8541484afbc9	241MB	66MB	U	
sonarqube:latest	6287ef13f526	2.53GB	1.09GB		
tomcat:latest	3f89144bff0c	589MB	158MB		
ubuntu:latest	6df9e8dd1eac	160MB	45.3MB		

```
ubuntu@ip-172-31-7-252:~$ docker rmi tomcat~
docker: command not found
ubuntu@ip-172-31-7-252:~$ docker rmi tomcat
Untagged: tomcat:latest
Deleted: sha256:3f89144bff0cb68ae4eecfe0a817cb319ac4f7f22beb03cd8d6fca0ef75f2dbf
ubuntu@ip-172-31-7-252:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA	In Use
jenkins/jenkins:lts	8547df3b0db2	801MB	290MB		
nginx:latest	8541484afbc9	241MB	66MB	U	
sonarqube:latest	6287ef13f526	2.53GB	1.09GB		
ubuntu:latest	6df9e8dd1eac	160MB	45.3MB		

```
ubuntu@ip-172-31-7-252:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS
869b56729b55	nginx	"/docker-entrypoint..."	nginx-container	13 minutes ago	Up 13 minutes

```
ubuntu@ip-172-31-7-252:~$
```

5. INSPECT JENKINS AND SONARQUBE IMAGES

Your assignment specifically asks:

Inspect the Jenkins image, SonarQube image.

Docker provides the docker inspect command for this.

Step 5.1 — Inspect Jenkins

Run:

```
docker inspect jenkins/jenkins:lts
```

This will produce a large JSON output.

You will see information such as:

Id

RepoTags

Created

Architecture

Os

Config

Env

ExposedPorts

Volumes

WorkingDir

Entrypoint

Cmd

```
ubuntu@ip-172-31-7-252: ~  
ubuntu@ip-172-31-7-252:~$ docker inspect jenkins/jenkins:lts  
[  
  {  
    "Id": "sha256:8547df3b0db2803d158ecc9499207a056bb30c23fddc18bb5b4a4dc14e77dd09",  
    "RepoTags": [  
      "jenkins/jenkins:lts"  
    ],  
    "RepoDigests": [  
      "jenkins/jenkins@sha256:8547df3b0db2803d158ecc9499207a056bb30c23fddc18bb5b4a4dc14e77dd09"  
    ],  
    "Comment": "buildkit.dockerfile.v0",  
    "Created": "2026-08-05T12:08:32.130140962Z",  
    "Config": {  
      "User": "jenkins",  
      "ExposedPorts": {  
        "50000/tcp": {},  
        "8080/tcp": {}  
      },  
      "Env": [  
        "PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",  
        "LANG=C.UTF-8",  
        "JENKINS_HOME=/var/jenkins_home",  
        "JENKINS_SLAVE_AGENT_PORT=50000",  
        "REF=/usr/share/jenkins/ref",  
        "JENKINS_UC=https://updates.jenkins.io",  
        "JENKINS_UC_EXPERIMENTAL=https://updates.jenkins.io/experimental",  
        "JENKINS_INCREMENTALS_REPO_MIRROR=https://repo.jenkins-ci.org/incremantals",  
        "COPY_REFERENCE_FILE_LOG=/var/jenkins_home/copy_reference_file.log",  
        "JAVA_HOME=/opt/java/openjdk",  
        "JENKINS_VERSION=2.568.2"  
      ],  
      "Entrypoint": [  
        "/usr/bin/tini",  
        "-",  
        "/usr/local/bin/jenkins.sh"  
      ],  
      "Volumes": {  
        "/var/jenkins_home": {}  
      },  
      "Labels": {  
        "org.opencontainers.image.description": "The Jenkins Continuous Integration and Delivery server",  
        "org.opencontainers.image.licenses": "MIT",  
        "org.opencontainers.image.revision": "53f5d618818bb7673c94b73e2e5335a9b5c5ed48",  
        "org.opencontainers.image.source": "https://github.com/jenkinsci/docker",  
        "org.opencontainers.image.title": "Official Jenkins Docker image",  
        "org.opencontainers.image.url": "https://www.jenkins.io/",  
        "org.opencontainers.image.vendor": "Jenkins project",  
        "org.opencontainers.image.version": "2.568.2"  
      }  
    }  
  ]  
]
```

Step 5.2 — Get specific Jenkins information

Instead of reading the entire output, you can use:

```
docker inspect --format='{{.Architecture}}' jenkins/jenkins:lts
```

Check exposed ports:

```
docker inspect --format='{{json .Config.ExposedPorts}}' jenkins/jenkins:lts
```

Check environment variables:

```
docker inspect --format='{{json .Config.Env}}' jenkins/jenkins:lts
```

Check volumes:

```
docker inspect --format='{{json .Config.Volumes}}' jenkins/jenkins:lts
```

Step 5.3 — Inspect SonarQube

Run:

```
docker inspect sonarqube:latest
```

You can also check exposed ports:

```
docker inspect --format='{{json .Config.ExposedPorts}}' sonarqube:latest
```

Check environment variables:

```
docker inspect --format='{{json .Config.Env}}' sonarqube:latest
```

```
ubuntu@ip-172-31-7-252: ~  
ubuntu@ip-172-31-7-252:~$ docker inspect sonarqube:latest  
[  
  {  
    "id": "sha256:6287ef13f5265d3eb266d5953ee08764d4ca73b3dabcd4e7de79ab040a6dfea",  
    "RepoTags": [  
      "sonarqube:latest"  
    ],  
    "RepoDigests": [  
      "sonarqube@sha256:6287ef13f5265d3eb266d5953ee08764d4ca73b3dabcd4e7de79ab040a6dfea"  
    ],  
    "Comment": "buildkit.dockerfile.v0",  
    "Created": "2026-08-18T20:29:07.683953233Z",  
    "Config": {  
      "User": "sonarqube",  
      "ExposedPorts": {  
        "9000/tcp": {}  
      },  
      "Env": [  
        "PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",  
        "JAVA_HOME=/opt/java/openjdk",  
        "LANG=en_US.UTF-8",  
        "LANGUAGE=en_US:en",  
        "LC_ALL=en_US.UTF-8",  
        "JAVA_VERSION=jdk-25.0.3+9",  
        "DOCKER_RUNNING=true",  
        "SONARQUBE_HOME=/opt/sonarqube",  
        "SONAR_VERSION=26.8.0.126808",  
        "SQ_DATA_DIR=/opt/sonarqube/data",  
        "SQ_EXTENSIONS_DIR=/opt/sonarqube/extensions",  
        "SQ_LOGS_DIR=/opt/sonarqube/logs",  
        "SQ_TEMP_DIR=/opt/sonarqube/temp",  
        "ES_TMPDIR=/opt/sonarqube/temp"  
      ],  
      "Entrypoint": [  
        "/opt/sonarqube/docker/entrypoint.sh"  
      ],  
      "Volumes": {  
        "/opt/sonarqube/data": {},  
        "/opt/sonarqube/extensions": {},  
        "/opt/sonarqube/logs": {},  
        "/opt/sonarqube/temp": {}  
      },  
      "WorkingDir": "/opt/sonarqube",  
      "Labels": {  
        "io.k8s.description": "SonarQube Community Build is a self-managed, automatic code review tool that systematically helps you deliver Clean Code.",  
        "io.openshift.min-cpu": "400m",  
        "io.openshift.min-memory": "2048M",  
        "io.openshift.non-scalable": "true",  
        "io.openshift.tags": "sonarqube,static-code-analysis,code-quality,clean-code",  
        "org.opencontainers.image.url": "https://github.com/SonarSource/docker-sonarqube",  
        "org.opencontainers.image.version": "24.04"  
      },  
      "StopSignal": "SIGINT"  
    },  
    "Architecture": "amd64",  
    "Os": "linux",  
    "Size": 1082947143,  
    "RootFS": {
```

What does docker inspect do?

docker inspect provides detailed low-level information about Docker objects such as images and containers. It returns configuration, networking, environment variables, storage, ports, architecture, IDs, and other metadata in JSON format

6. RUN JENKINS CONTAINER AND EXECUTE A SAMPLE JOB

This is the biggest part of your assignment.

Step 6.1 — Run Jenkins

Jenkins uses port:

8080

inside the container.

Run:

```
docker run -d \
  --name jenkins-container \
  -p 8080:8080 \
  -p 50000:50000 \
  jenkins/jenkins:lts
```

Explanation:

8080:8080

means:

EC2 port 8080



Jenkins container port 8080

And:

50000:50000

is used for Jenkins inbound agent communication.

Step 6.2 — Check Jenkins container

docker ps

You should see something similar to:

CONTAINER ID	IMAGE	PORTS
xxxxxxx	jenkins/jenkins:lts	0.0.0.0:8080->8080/tcp

```
ubuntu@ip-172-31-7-252:~$ docker run -d \
  --name jenkins-container \
  -p 8080:8080 \
  -p 50000:50000 \
  jenkins/jenkins:lts
751e5087fedc4280baddb7442a703ec2113ad17783788a72fb87eb752e3fa
ubuntu@ip-172-31-7-252:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
751e5087fedc   jenkins/jenkins:lts   "/usr/bin/tini -- /u..." 17 seconds ago   Up           0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000
tcp, [::]:50000->50000/tcp   jenkins-container
869b56729b55   nginx          "/docker-entrypoint..." 24 minutes ago   Up           0.0.0.0:81->80/tcp, [::]:81->80/tcp
nginx-container
ubuntu@ip-172-31-7-252:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                                     NAMES
751e5087fedc   jenkins/jenkins:lts   "/usr/bin/tini -- /u..." 48 seconds ago   Up 48 seconds   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp   jenkins-container
869b56729b55   nginx          "/docker-entrypoint..." 25 minutes ago   Up 25 minutes   0.0.0.0:81->80/tcp, [::]:81->80/tcp                                     nginx-container
ubuntu@ip-172-31-7-252:~$
```

Step 6.3 — Get Jenkins Initial Password

Jenkins will generate an initial administrator password.

Run:

```
docker exec jenkins-container cat /var/jenkins_home/secrets/initialAdminPassword
```

You will get something like:

```
a8f7xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Copy this password.

Step 6.4 — Open Jenkins

First add port **8080** to your EC2 Security Group.

Go to:

EC2 → Security Groups → Inbound rules → Edit inbound rules

Add:

Type: Custom TCP

Port: 8080

Source: My IP

Save.

Now open:

```
http://YOUR_PUBLIC_IP:8080
```

For example:

```
http://54.123.45.67:8080
```

You should see:

Unlock Jenkins

Paste the password obtained from:

```
docker exec jenkins-container cat /var/jenkins_home/secrets/initialAdminPassword
```

Click:

Continue

Step 6.5 — Install Jenkins Plugins

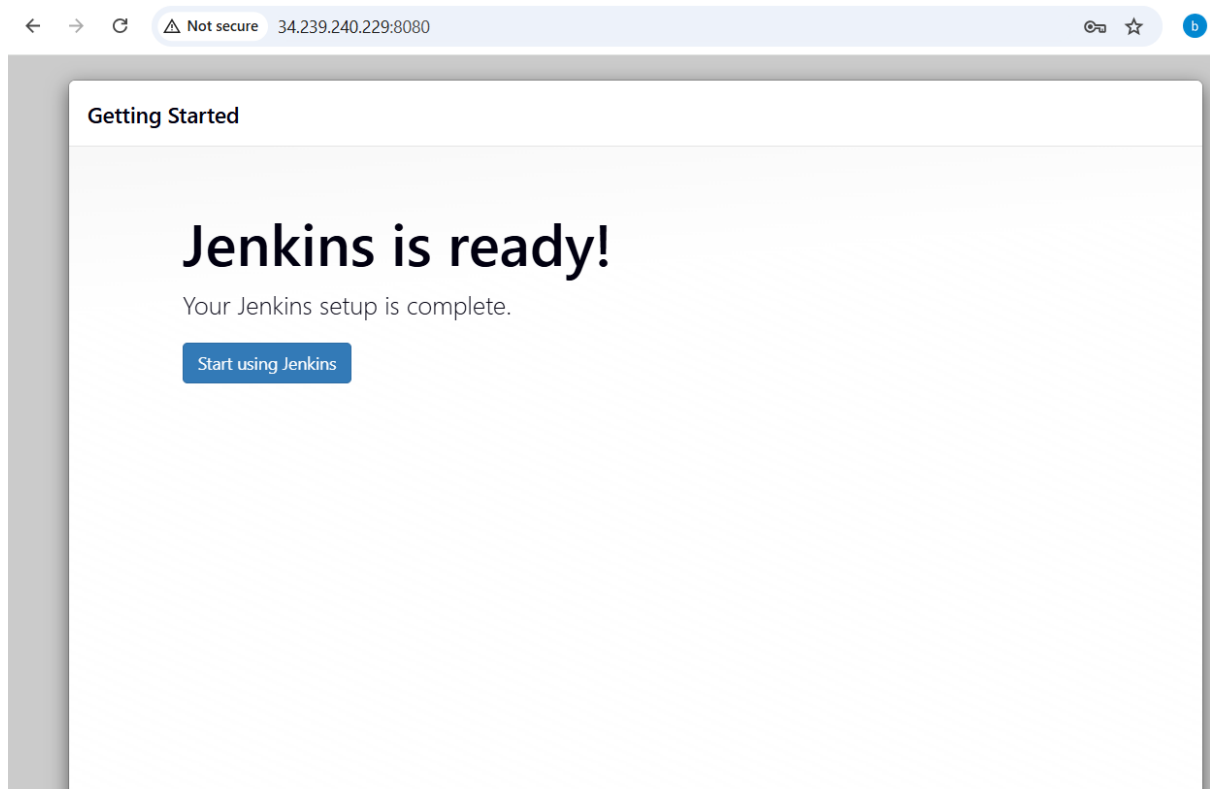
Jenkins will ask:

Customize Jenkins

Select:

Install suggested plugins

Wait for the installation to complete.



Step 6.5 — Install Jenkins Plugins

Jenkins will ask:

Customize Jenkins

Select:

Install suggested plugins

Wait for the installation to complete.

Step 6.6 — Create Jenkins Admin User

Create your administrator account.

For example:

Username:

admin

Password:

Your secure password

Full name:

Rajkumari

Email:

Your email

Then click:

Save and Continue

Step 6.7 — Create Sample Jenkins Job

From the Jenkins dashboard:

New Item

Enter:

Name:

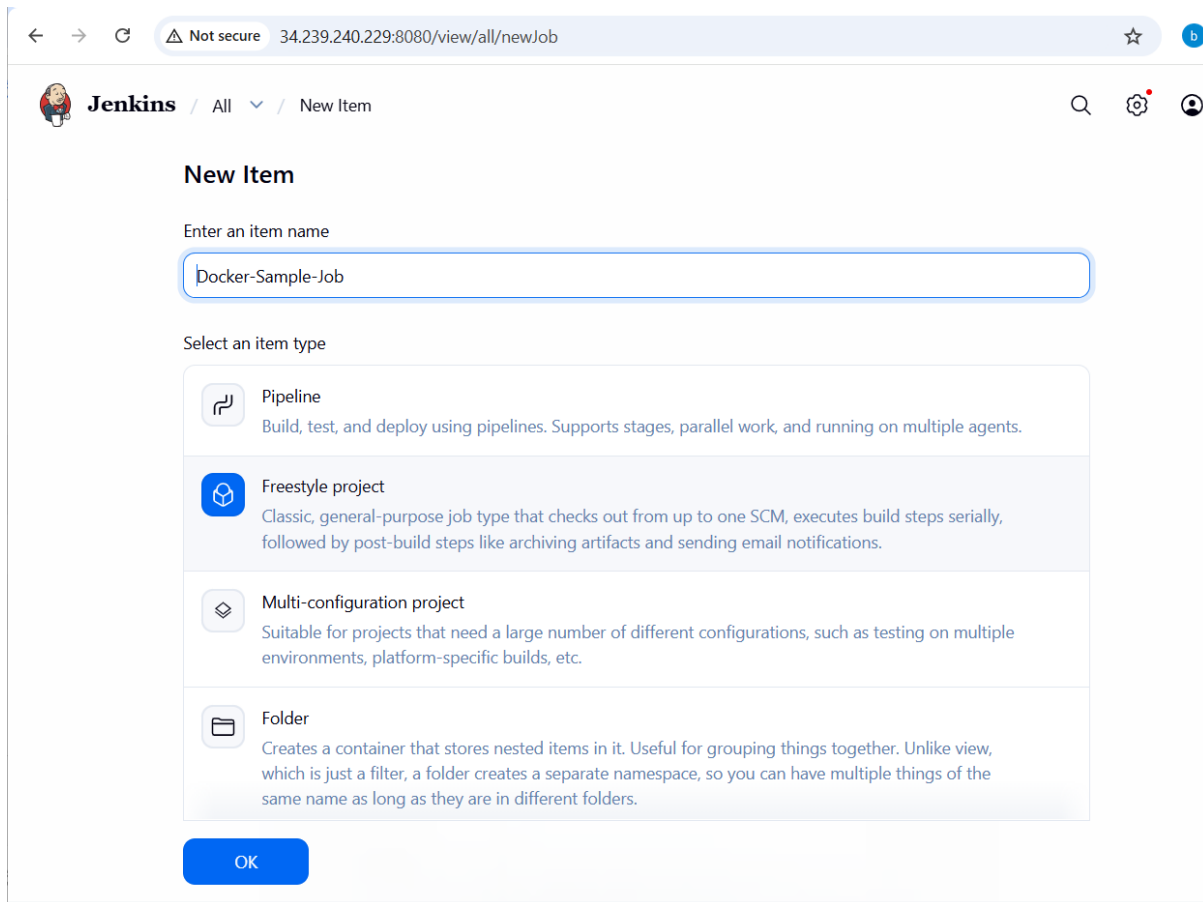
Docker-Sample-Job

Select:

Freestyle project

Click:

OK



Step 6.8 — Add a Sample Build Command

Go to:

Build Steps → Add build step → Execute shell

← → ↻ ⚠ Not secure 34.239.240.229:8080/job/Docker-Sample-Job/configure ☆ b

Jenkins / Docker-Sample-Job ▾ / Configure 🔍 ⚙️ 👤

Configure

- ⚙️ General
- 🔑 Source Code Management
- 🕒 Triggers
- 🌐 Environment**
- 📋 Build Steps
- 📦 Post-build Actions

☐ Inspect build log for published build scans

☐ Terminate a build if it's stuck

☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

⚙️ Execute shell ?

✖

Command

See [the list of available environment variables](#)

```
echo "Build Number:"  
echo $BUILD_NUMBER  
  
echo "Job Name:"  
echo $JOB_NAME
```

Advanced ▾

Save

Apply

Step 6.9 — Run the Job

On the Jenkins job page, click:

Build Now

← → ↻ ⚠ Not secure 34.239.240.229:8080/job/Docker-Sample-Job/ ☆ b ⋮

Jenkins / Docker-Sample-Job 🔍 ⚙️ 👤

Status

</> Changes

📁 Workspace

▶ Build Now

🗑 Delete Project

⚙️ Configure

✎ Rename

✅ Docker-Sample-Job

Permalinks

- [Last build \(#2\), 14 min ago](#)
- [Last stable build \(#2\), 14 min ago](#)
- [Last successful build \(#2\), 14 min ago](#)
- [Last completed build \(#2\), 14 min ago](#)

Builds > ⋮

🔍 Filter /

Today

✅ #2 9:21 AM ▾

✅ #1 9:20 AM ▾

← → ↻ Not secure 34.239.240.229:8080/job/Docker-Sample-Job/2/console ☆ ⓘ

 **Jenkins** / Docker-Sample-Job ▾ / #2 ▾ 🔍 ⚙️ 👤

← Previous Build

```
+ echo Docker Jenkins Sample Job
Docker Jenkins Sample Job
+ echo =====
+ echo Hello from Jenkins!
Hello from Jenkins!
+ echo Current user:
Current user:
+ whoami
jenkins
+ echo Current date:
Current date:
+ date
Wed Aug 19 09:21:14 UTC 2026
+ echo Jenkins workspace:
Jenkins workspace:
+ pwd
/var/jenkins_home/workspace/Docker-Sample-Job
+ echo Build Number:
Build Number:
+ echo 2
2
+ echo Job Name:
Job Name:
+ echo Docker-Sample-Job
Docker-Sample-Job
Finished: SUCCESS
```